1. Problem Title & Link

Title: **LeetCode 881 : Boats to Save People**

Link: https://leetcode.com/problems/boats-to-save-people/

2. Problem Statement (Clear Explanation)

You are given an array people where:

people[i]

represents the **weight of the ith person**.

You are also given an integer:

limit

which represents the **maximum weight a boat can carry**.

Each boat can carry **at most two people at the same time**, and the **sum of their weights must not exceed the boat's weight limit**.

Your task is to determine the **minimum number of boats required to rescue all people**.

Important rules:

- Each boat can carry **1 or 2 people only**.
- The **combined weight** of people in a boat must be **≤ limit**.
- Every person must be placed in **exactly one boat**.

Return the **minimum number of boats required**.

3. Examples (Input → Output)

Example 1

Input

people = [1,2]

limit = 3

Output

1

Explanation

Both people can share the same boat:

1 + 2 = 3 ≤ limit

Example 2

Input

people = [3,2,2,1]

limit = 3

Output

3

Explanation

Possible arrangement:

1 + 2 → boat 1

2 → boat 2

3 → boat 3

Total boats required = **3**

Example 3

Input

people = [3,5,3,4]

limit = 5

Output

4

Explanation

Each heavy person must go alone:

3

5

3

4

Total boats = **4**

4. Constraints

1 ≤ people.length ≤ 50000

1 ≤ people[i] ≤ limit ≤ 30000

Additional notes:

- Each boat can carry **at most two people**
- Every person must be rescued
- The goal is to **minimize the number of boats**

5. Core Concept (Pattern / Topic)

Greedy

Two Pointers

6. Thought Process (Step-by-Step Explanation)

Step 1 — Understand the objective

We want to **minimize the number of boats**.

Each boat can carry **two people if their combined weight ≤ limit**.

Step 2 — Greedy Insight

To minimize boats:

- Pair the **lightest person with the heaviest person** whenever possible.

Why?

If the heaviest person cannot pair with the lightest person, they **cannot pair with anyone else**.

Therefore, they must go **alone**.

Step 3 — Strategy

1. Sort the people array
2. Use **two pointers**:
    - left → lightest person
    - right → heaviest person
3. If:

people[left] + people[right] ≤ limit

Then they share a boat.

Otherwise:

heaviest person goes alone

7. Visual / Intuition Diagram

Example

people = [3,2,2,1]

limit = 3

Sorted:

[1,2,2,3]

Two-pointer process:

1 + 3 > 3 → 3 alone

1 + 2 ≤ 3 → pair

2 alone

Boats = **3**

8. Pseudocode (Language Independent)

```
function numRescueBoats(people, limit):

  sort(people)

  left = 0
  right = len(people) - 1
```

```
    boats = 0

    while left <= right:

        if people[left] + people[right] <= limit:
            left += 1

        right -= 1
        boats += 1

    return boats
```

9. Code Implementation

Python

```
def numRescueBoats(people, limit):

    people.sort()

    left = 0
    right = len(people) - 1

    boats = 0

    while left <= right:

        if people[left] + people[right] <= limit:
            left += 1

        right -= 1
        boats += 1

    return boats
```

Java

```
import java.util.*;
```

```
class Solution {

    public int numRescueBoats(int[] people, int limit) {

        Arrays.sort(people);

        int left = 0;
        int right = people.length - 1;
        int boats = 0;

        while(left <= right) {

            if(people[left] + people[right] <= limit) {
                left++;
            }

            right--;
            boats++;
        }

        return boats;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n log n)

Sorting dominates runtime.

Space Complexity

O(1)

Only pointers used.

11. Common Mistakes / Edge Cases

Common mistakes:

1. Trying to pair **two heavy people**
2. Using brute force pairing
3. Not sorting the array first

Edge cases:

people = [5]

limit = 5

Answer = **1**

12. Detailed Dry Run

Example

people = [3,2,2,1]

limit = 3

Sorted:

[1,2,2,3]

Left	Right	Sum	Action	Boats
1	3	4	3 alone	1
1	2	3	pair	2
2	2	2	alone	3

Result:

3 boats

13. Common Use Cases (Real-Life / Interview)

Similar patterns appear in:

- **Resource allocation problems**
- **Transportation optimization**
- **Packing optimization**

14. Common Traps (Important!)

1. Using **nested loops ($O(n^2)$)**
2. Not pairing **lightest with heaviest**
3. Incorrect pointer movement

15. Builds To (Related LeetCode Problems)

Progression:

LC 167 Two Sum II

LC 881 Boats to Save People

LC 259 3Sum Smaller

LC 15 3Sum

LC 42 Trapping Rain Water

16. Alternate Approaches + Comparison

Approach	Time	Notes
Greedy + Two Pointers	O(n log n)	Optimal
Brute Force Pairing	$O(n^2)$	Too slow

17. Why This Solution Works (Short Intuition)

If the **heaviest person cannot pair with the lightest**, they cannot pair with anyone else, so sending them alone is always optimal.

18. Variations / Follow-Up Questions

Possible interview extensions:

1. Boats can carry **3 people**
2. Return the **actual boat assignments**
3. Boats have **different weight limits**
4. Minimize **total trips instead of boats**.
5.